

INFO145 - Diseño y Análisis de Algoritmos

5 - Ecuaciones de Recurrencias

Héctor Ferrada
académico

Instituto de Informática
Universidad Austral de Chile

Ecuaciones de Recurrencias

- Son del tipo de ecuaciones que expresan el tiempo de ejecución de un algoritmo de forma recursiva.
- Generalmente la recursividad es en términos de un input más pequeño.

Ecuaciones de Recurrencias

- Son del tipo de ecuaciones que expresan el tiempo de ejecución de un algoritmo de forma recursiva.
- Generalmente la recursividad es en términos de un input más pequeño.
- Existen muchas formas de soluciones recurrentes, algunos ejemplos son:

$$T(n) = O(n) + 2T(n/2)$$

→ ¿Algoritmo conocido?

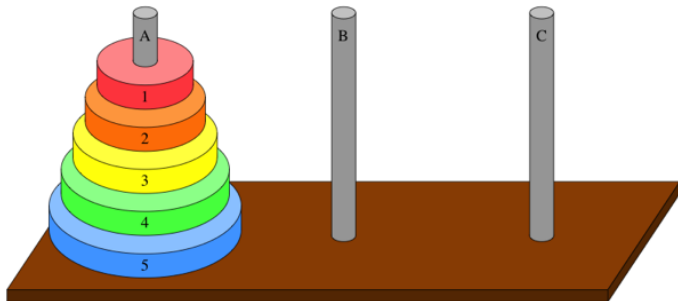
Ecuaciones de Recurrencias

- Son del tipo de ecuaciones que expresan el tiempo de ejecución de un algoritmo de forma recursiva.
- Generalmente la recursividad es en términos de un input más pequeño.
- Existen muchas formas de soluciones recurrentes, algunos ejemplos son:

$$T(n) = O(n) + 2T(n/2) \quad \rightarrow \text{¿Algoritmo conocido?}$$

$$H(n) = H(n-1) + O(1) + H(n-1) = 2H(n-1) + O(1) \\ \rightarrow \text{¿Algoritmo?}$$

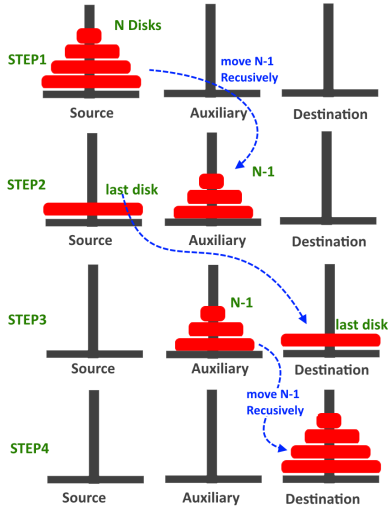
Torres de Hanói



El problema de las Torres de Hanói, se define como:

$\mathcal{H}(n)$: mover los n discos de la torre A a la torre C, con la condición de que solo es posible mover 1 disco a la vez dejándolo sobre un disco de mayor tamaño, o en una torre vacía.

Algoritmo de Hanói



Algoritmo de Hanói

Algoritmo $\mathcal{H}(n) = \text{hanoi}(n, A, B, C)$ mueve los n discos desde X a Z .

Input: 3 stacks A , B y C de capacidad n . A tiene los n discos

Output: C con los n discos en orden

```
hanoi( $n, A, B, C$ ){  
    if( $n \geq 1$ ) then  
        hanoi( $n - 1, A, C, B$ )  
         $C.\text{push}(A.\text{pop})$   
        hanoi( $n - 1, B, A, C$ )  
}
```

Algoritmo de Hanói

Algoritmo $\mathcal{H}(n) = \text{hanoi}(n, A, B, C)$ mueve los n discos desde X a Z .

Input: 3 stacks A , B y C de capacidad n . A tiene los n discos

Output: C con los n discos en orden

```
hanoi( $n, A, B, C$ ){  
    if( $n \geq 1$ ) then  
        hanoi( $n - 1, A, C, B$ )  
         $C.\text{push}(A.\text{pop})$   
        hanoi( $n - 1, B, A, C$ )  
}
```

$$\Rightarrow \mathcal{H}(n) = 1 + 2\mathcal{H}(n - 1)$$

Algoritmo de Hanói

Algoritmo $\mathcal{H}(n) = \text{hanoi}(n, A, B, C)$ mueve los n discos desde X a Z .

Input: 3 stacks A , B y C de capacidad n . A tiene los n discos

Output: C con los n discos en orden

```
hanoi( $n, A, B, C$ ){  
    if( $n \geq 1$ ) then  
        hanoi( $n - 1, A, C, B$ )  
         $C.\text{push}(A.\text{pop})$   
        hanoi( $n - 1, B, A, C$ )  
}
```

$\Rightarrow \mathcal{H}(n) = 1 + 2\mathcal{H}(n - 1) \Rightarrow$ Resolver y Demostrar por Inducción:

Algoritmo de Hanói

Algoritmo $\mathcal{H}(n) = \text{hanoi}(n, A, B, C)$ mueve los n discos desde X a Z .

Input: 3 stacks A , B y C de capacidad n . A tiene los n discos

Output: C con los n discos en orden

```
hanoi( $n, A, B, C$ ){  
    if( $n \geq 1$ ) then  
        hanoi( $n - 1, A, C, B$ )  
         $C.\text{push}(A.\text{pop})$   
        hanoi( $n - 1, B, A, C$ )  
}
```

$\Rightarrow \mathcal{H}(n) = 1 + 2\mathcal{H}(n - 1) \Rightarrow$ Resolver y Demostrar por Inducción:

$$\mathcal{H}(n - 1) = 1 + 2\mathcal{H}(n - 2)$$

$$\mathcal{H}(n - 2) = 1 + 2\mathcal{H}(n - 3)$$

Algoritmo de Hanói

Algoritmo $\mathcal{H}(n) = \text{hanoi}(n, A, B, C)$ mueve los n discos desde X a Z .

Input: 3 stacks A , B y C de capacidad n . A tiene los n discos

Output: C con los n discos en orden

```
hanoi( $n, A, B, C$ ){  
    if( $n \geq 1$ ) then  
        hanoi( $n - 1, A, C, B$ )  
         $C.\text{push}(A.\text{pop})$   
        hanoi( $n - 1, B, A, C$ )  
}
```

$\Rightarrow \mathcal{H}(n) = 1 + 2\mathcal{H}(n - 1) \Rightarrow$ Resolver y Demostrar por Inducción:

$$\mathcal{H}(n - 1) = 1 + 2\mathcal{H}(n - 2)$$

$$\mathcal{H}(n - 2) = 1 + 2\mathcal{H}(n - 3)$$

$$\mathcal{H}(n - 1) = 1 + 2(1 + 2\mathcal{H}(n - 3))$$

Algoritmo de Hanói

Algoritmo $\mathcal{H}(n) = \text{hanoi}(n, A, B, C)$ mueve los n discos desde X a Z .

Input: 3 stacks A , B y C de capacidad n . A tiene los n discos

Output: C con los n discos en orden

```
hanoi( $n, A, B, C$ ){  
    if( $n \geq 1$ ) then  
        hanoi( $n - 1, A, C, B$ )  
         $C.\text{push}(A.\text{pop})$   
        hanoi( $n - 1, B, A, C$ )  
}
```

$\Rightarrow \mathcal{H}(n) = 1 + 2\mathcal{H}(n - 1) \Rightarrow$ Resolver y Demostrar por Inducción:

$$\mathcal{H}(n - 1) = 1 + 2\mathcal{H}(n - 2)$$

$$\mathcal{H}(n - 2) = 1 + 2\mathcal{H}(n - 3)$$

$$\begin{aligned}\mathcal{H}(n - 1) &= 1 + 2(1 + 2\mathcal{H}(n - 3)) \\ &= 1 + 2 + 2^2\mathcal{H}(n - 3)\end{aligned}$$

Algoritmo de Hanói

Algoritmo $\mathcal{H}(n) = \text{hanoi}(n, A, B, C)$ mueve los n discos desde X a Z .

Input: 3 stacks A , B y C de capacidad n . A tiene los n discos

Output: C con los n discos en orden

```
hanoi( $n, A, B, C$ ){  
    if( $n \geq 1$ ) then  
        hanoi( $n - 1, A, C, B$ )  
         $C.\text{push}(A.\text{pop})$   
        hanoi( $n - 1, B, A, C$ )  
}
```

$\Rightarrow \mathcal{H}(n) = 1 + 2\mathcal{H}(n - 1) \Rightarrow$ Resolver y Demostrar por Inducción:

$$\mathcal{H}(n - 1) = 1 + 2\mathcal{H}(n - 2)$$

$$\mathcal{H}(n - 2) = 1 + 2\mathcal{H}(n - 3)$$

$$\mathcal{H}(n - 1) = 1 + 2(1 + 2\mathcal{H}(n - 3))$$

$$= 1 + 2 + 2^2\mathcal{H}(n - 3)$$

$$\Rightarrow \mathcal{H}(n) = 1 + 2 + 2^2 + 2^3\mathcal{H}(n - 3)$$

Algoritmo de Hanói

Algoritmo $\mathcal{H}(n) = \text{hanoi}(n, A, B, C)$ mueve los n discos desde X a Z .

Input: 3 stacks A , B y C de capacidad n . A tiene los n discos

Output: C con los n discos en orden

```
hanoi( $n, A, B, C$ ){  
    if( $n \geq 1$ ) then  
        hanoi( $n - 1, A, C, B$ )  
         $C.\text{push}(A.\text{pop})$   
        hanoi( $n - 1, B, A, C$ )  
}
```

$\Rightarrow \mathcal{H}(n) = 1 + 2\mathcal{H}(n - 1) \Rightarrow$ Resolver y Demostrar por Inducción:

$$\mathcal{H}(n - 1) = 1 + 2\mathcal{H}(n - 2)$$

$$\mathcal{H}(n - 2) = 1 + 2\mathcal{H}(n - 3)$$

$$\begin{aligned}\mathcal{H}(n - 1) &= 1 + 2(1 + 2\mathcal{H}(n - 3)) \\ &= 1 + 2 + 2^2\mathcal{H}(n - 3)\end{aligned}$$

$$\Rightarrow \mathcal{H}(n) = 1 + 2 + 2^2 + 2^3\mathcal{H}(n - 3)$$

Generalizando, **supondremos** que se cumple:

$$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^{k-1} + 2^k\mathcal{H}(n - k)$$

Algoritmo de Hanói

$$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^{k-1} + 2^k \mathcal{H}(n - k)$$

Si $k = n$, entonces:

$$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^n \mathcal{H}(0)$$

Algoritmo de Hanói

$$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^{k-1} + 2^k \mathcal{H}(n - k)$$

Si $k = n$, entonces:

$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^n \mathcal{H}(0)$, acá $\mathcal{H}(0) = 1$, es constante en un paso

Algoritmo de Hanói

$$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^{k-1} + 2^k \mathcal{H}(n - k)$$

Si $k = n$, entonces:

$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^n \mathcal{H}(0)$, acá $\mathcal{H}(0) = 1$, es constante en un paso

$$\Rightarrow \mathcal{H}(n) = \sum_{k=0}^n 2^k$$

Algoritmo de Hanói

$$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^{k-1} + 2^k \mathcal{H}(n - k)$$

Si $k = n$, entonces:

$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^n \mathcal{H}(0)$, acá $\mathcal{H}(0) = 1$, es constante en un paso

$$\Rightarrow \mathcal{H}(n) = \sum_{k=0}^n 2^k = \frac{1 - 2^{n+1}}{1 - 2} + 2^n = 2^{n+1} - 1$$

Recuerde que

$$\sum_{i=0}^{k-1} a^i = \frac{1 - a^k}{1 - a}$$

Algoritmo de Hanói

$$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^{k-1} + 2^k \mathcal{H}(n - k)$$

Si $k = n$, entonces:

$\mathcal{H}(n) = 1 + 2 + 2^2 + 2^3 + \dots + 2^n \mathcal{H}(0)$, acá $\mathcal{H}(0) = 1$, es constante en un paso

$$\Rightarrow \mathcal{H}(n) = \sum_{k=0}^n 2^k = \frac{1 - 2^{n+1}}{1 - 2} + 2^n = 2^{n+1} - 1$$

Recuerde que

$$\sum_{i=0}^{k-1} a^i = \frac{1 - a^k}{1 - a}$$

Por tanto: $\mathcal{H}(n) = 2^{n+1} - 1$

Algoritmo de Hanói

Demostrar por inducción que $\mathcal{H}(n) = 2^{n+1} - 1$

Algoritmo de Hanói

Demostrar por inducción que $\mathcal{H}(n) = 2^{n+1} - 1$

Dem.: i- Paso base para $n = 1$, $\mathcal{H}(1) = 2^{1+1} - 1 = 3$

Algoritmo de Hanói

Demostrar por inducción que $\mathcal{H}(n) = 2^{n+1} - 1$

Dem.: i- Paso base para $n = 1$, $\mathcal{H}(1) = 2^{1+1} - 1 = 3$
y por definición, $\mathcal{H}(1) = 1 + 2\mathcal{H}(1 - 1)$

Algoritmo de Hanói

Demostrar por inducción que $\mathcal{H}(n) = 2^{n+1} - 1$

Dem.: i- Paso base para $n = 1$, $\mathcal{H}(1) = 2^{1+1} - 1 = 3$

y por definición, $\mathcal{H}(1) = 1 + 2\mathcal{H}(1 - 1) = 1 + 2\mathcal{H}(0) = 3 \Rightarrow$ Se cumple.

Algoritmo de Hanói

Demostrar por inducción que $\mathcal{H}(n) = 2^{n+1} - 1$

Dem.: i- Paso base para $n = 1$, $\mathcal{H}(1) = 2^{1+1} - 1 = 3$
y por definición, $\mathcal{H}(1) = 1 + 2\mathcal{H}(1 - 1) = 1 + 2\mathcal{H}(0) = 3 \Rightarrow$ Se cumple.

Dem.: ii- Paso inductivo para $n + 1$,
 $\mathcal{H}(n + 1) = 1 + 2\mathcal{H}(n)$

Algoritmo de Hanói

Demostrar por inducción que $\mathcal{H}(n) = 2^{n+1} - 1$

Dem.: i- Paso base para $n = 1$, $\mathcal{H}(1) = 2^{1+1} - 1 = 3$
y por definición, $\mathcal{H}(1) = 1 + 2\mathcal{H}(1 - 1) = 1 + 2\mathcal{H}(0) = 3 \Rightarrow$ Se cumple.

Dem.: ii- Paso inductivo para $n + 1$,
 $\mathcal{H}(n + 1) = 1 + 2\mathcal{H}(n) = 1 + 2(2^{n+1} - 1)$

Algoritmo de Hanói

Demostrar por inducción que $\mathcal{H}(n) = 2^{n+1} - 1$

Dem.: i- Paso base para $n = 1$, $\mathcal{H}(1) = 2^{1+1} - 1 = 3$
y por definición, $\mathcal{H}(1) = 1 + 2\mathcal{H}(1 - 1) = 1 + 2\mathcal{H}(0) = 3 \Rightarrow$ Se cumple.

Dem.: ii- Paso inductivo para $n + 1$,
 $\mathcal{H}(n + 1) = 1 + 2\mathcal{H}(n) = 1 + 2(2^{n+1} - 1) = 1 + 2 \cdot 2^{n+1} - 2$

Algoritmo de Hanói

Demostrar por inducción que $\mathcal{H}(n) = 2^{n+1} - 1$

Dem.: i- Paso base para $n = 1$, $\mathcal{H}(1) = 2^{1+1} - 1 = 3$
y por definición, $\mathcal{H}(1) = 1 + 2\mathcal{H}(1 - 1) = 1 + 2\mathcal{H}(0) = 3 \Rightarrow$ Se cumple.

Dem.: ii- Paso inductivo para $n + 1$,
 $\mathcal{H}(n + 1) = 1 + 2\mathcal{H}(n) = 1 + 2(2^{n+1} - 1) = 1 + 2 \cdot 2^{n+1} - 2 = 2^{n+2} - 1$

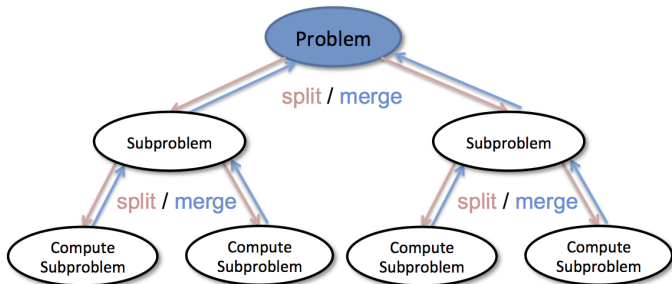
Por i- y por ii- Se demuestra que $\mathcal{H}(n) = 2^{n+1} - 1$

Técnica de Diseño Divide y Reinaras

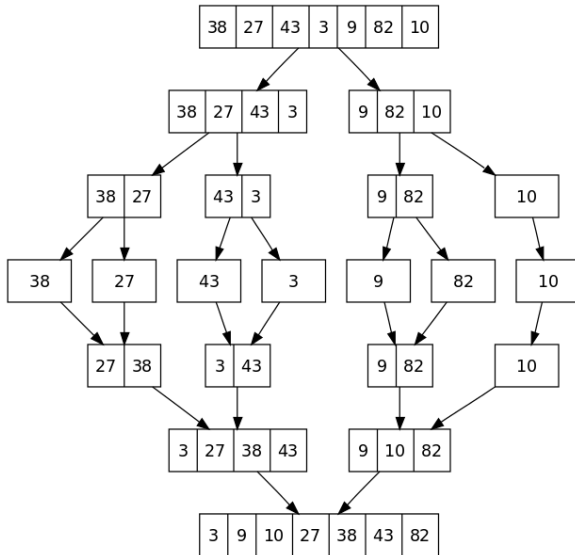
Analizaremos el algoritmo **Merge Sort** que utiliza esta técnica.

Divide y Reinaras se resume en:

1. Divide el problema original en 2 o más subproblemas similares al problema original, pero de menor tamaño.
2. Cada subproblema se invoca recursivamente.
3. El último paso es combinar las soluciones locales de cada subproblema para formar la solución global del problema original.



Algoritmo Merge Sort



Algoritmo Merge Sort

Algoritmo $\text{mergeSort}(A, \ell, r)$: ordena ascendentemente $A[\ell..r]$.

Input: Arreglo $A[1..n]$, y sus límites $\ell = 1$ y $r = n$

Output: A ordenado ascendentemente

```
mergeSort( $A, \ell, r$ ) {  
    if( $r - \ell \geq 1$ ) then  
         $m = (\ell + r)/2$   
        mergeSort( $A, \ell, m$ )            $\rightarrow T(n/2)$   
        mergeSort( $A, m + 1, r$ )        $\rightarrow T(n/2)$   
        merge( $A, \ell, m, r$ )            $\rightarrow \Theta(g(n)) = ?$   
}
```

Algoritmo Merge Sort

Algoritmo $\text{mergeSort}(A, \ell, r)$: ordena ascendentemente $A[\ell..r]$.

Input: Arreglo $A[1..n]$, y sus límites $\ell = 1$ y $r = n$

Output: A ordenado ascendentemente

```
mergeSort( $A, \ell, r$ ) {  
    if ( $r - \ell \geq 1$ ) then  
         $m = (\ell + r) / 2$   
        mergeSort( $A, \ell, m$ )            $\rightarrow T(n/2)$   
        mergeSort( $A, m + 1, r$ )        $\rightarrow T(n/2)$   
        merge( $A, \ell, m, r$ )            $\rightarrow \Theta(g(n)) = ?$   
}  
 $\Rightarrow T(n) = \Theta(g(n)) + 2T(n/2)$ 
```

Algoritmo Merge Sort

```
merge( $A, \ell, m, r$ ){  
     $n_1 = m - \ell + 1$   
     $n_2 = r - m$   
    Sean:  $L[1..n_1 + 1]$  y  $R[1..n_2 + 1]$  nuevos arreglos  
    for  $i = 1$  to  $n_1$   
         $L[i] = A[\ell + i - 1]$   
    for  $j = 1$  to  $n_2$   
         $R[j] = A[m + j]$   
     $L[n_1] = R[n_2] = \infty$   
     $i = j = 1$   
    for  $k = \ell$  to  $r$   
        if ( $L[i] \leq R[j]$ ) then  
             $A[k] = L[i]$   
             $i = i + 1$   
        else  
             $A[k] = R[j]$   
             $j = j + 1$   
}
```

Algoritmo Merge Sort

```
merge( $A, \ell, m, r$ ){  
     $n_1 = m - \ell + 1$   
     $n_2 = r - m$   
    Sean:  $L[1..n_1 + 1]$  y  $R[1..n_2 + 1]$  nuevos arreglos  $\rightarrow O(n)$   
    for  $i = 1$  to  $n_1$   $\rightarrow \Theta(n_1) = O(n)$   
         $L[i] = A[\ell + i - 1]$   
    for  $j = 1$  to  $n_2$   $\rightarrow \Theta(n_2) = O(n)$   
         $R[j] = A[m + j]$   
     $L[n_1] = R[n_2] = \infty$   
     $i = j = 1$   
    for  $k = \ell$  to  $r$   $\rightarrow \Theta(n)$   
        if ( $L[i] \leq R[j]$ ) then  
             $A[k] = L[i]$   
             $i = i + 1$   
        else  
             $A[k] = R[j]$   
             $j = j + 1$   
}
```

Algoritmo Merge Sort

```
merge( $A, \ell, m, r$ ) {  
     $n_1 = m - \ell + 1$   
     $n_2 = r - m$   
    Sean:  $L[1..n_1 + 1]$  y  $R[1..n_2 + 1]$  nuevos arreglos  $\rightarrow O(n)$   
    for  $i = 1$  to  $n_1$   $\rightarrow \Theta(n_1) = O(n)$   
         $L[i] = A[\ell + i - 1]$   
    for  $j = 1$  to  $n_2$   $\rightarrow \Theta(n_2) = O(n)$   
         $R[j] = A[m + j]$   
     $L[n_1] = R[n_2] = \infty$   
     $i = j = 1$   
    for  $k = \ell$  to  $r$   $\rightarrow \Theta(n)$   
        if ( $L[i] \leq R[j]$ ) then  
             $A[k] = L[i]$   
             $i = i + 1$   
        else  
             $A[k] = R[j]$   
             $j = j + 1$   
    }  
     $\Rightarrow \text{merge}(A, \ell, m, r) = \Theta(r - \ell) = \Theta(n)$ 
```

Algoritmo Merge Sort

```
merge( $A, \ell, m, r$ ) {  
     $n_1 = m - \ell + 1$   
     $n_2 = r - m$   
    Sean:  $L[1..n_1 + 1]$  y  $R[1..n_2 + 1]$  nuevos arreglos  $\rightarrow O(n)$   
    for  $i = 1$  to  $n_1$   $\rightarrow \Theta(n_1) = O(n)$   
         $L[i] = A[\ell + i - 1]$   
    for  $j = 1$  to  $n_2$   $\rightarrow \Theta(n_2) = O(n)$   
         $R[j] = A[m + j]$   
     $L[n_1] = R[n_2] = \infty$   
     $i = j = 1$   
    for  $k = \ell$  to  $r$   $\rightarrow \Theta(n)$   
        if ( $L[i] \leq R[j]$ ) then  
             $A[k] = L[i]$   
             $i = i + 1$   
        else  
             $A[k] = R[j]$   
             $j = j + 1$   
    }  
⇒ merge( $A, \ell, m, r$ ) =  $\Theta(r - \ell) = \Theta(n)$   
∴ mergeSort( $n$ )  $\rightarrow T(n) = \Theta(n) + 2T(n/2)$ 
```

Merge Sort Tree

Calcularemos el costo de $T(n)$ mediante el árbol de recurrencia del algoritmo.

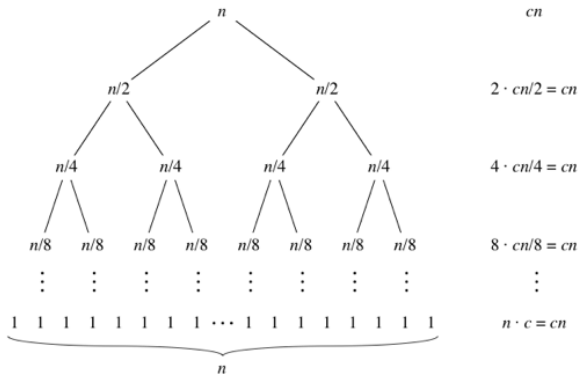
Árbol de Recurrencia¹ del algoritmo mergeSort:

¹Técnica utilizada para determinar costos en algoritmos recursivos.

Merge Sort Tree

Calcularemos el costo de $T(n)$ mediante el árbol de recurrencia del algoritmo.

Árbol de Recurrencia¹ del algoritmo mergeSort:

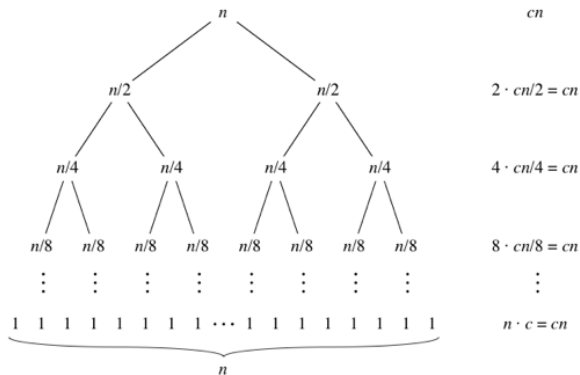


¹Técnica utilizada para determinar costos en algoritmos recursivos.

Merge Sort Tree

Calcularemos el costo de $T(n)$ mediante el árbol de recurrencia del algoritmo.

Árbol de Recurrencia¹ del algoritmo mergeSort:



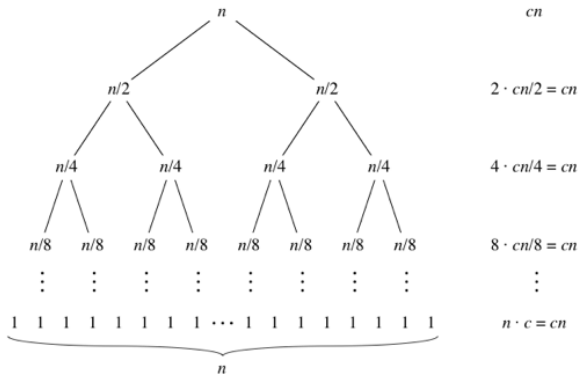
Note que la altura del árbol es $h = \lceil \lg n \rceil$

¹Técnica utilizada para determinar costos en algoritmos recursivos.

Merge Sort Tree

Calcularemos el costo de $T(n)$ mediante el árbol de recurrencia del algoritmo.

Árbol de Recurrencia¹ del algoritmo mergeSort:



Note que la altura del árbol es $h = \lceil \lg n \rceil$

$\Rightarrow \text{mergeSort}(n) = T(n) = h \cdot c \cdot n = \Theta(n \log n)$

¹Técnica utilizada para determinar costos en algoritmos recursivos.

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2 T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2 T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$T(n) = T(2^k) = c \cdot 2^k + 2 T(2^k/2); \text{ con } T(1) = c'$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2 T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2 T(2^k/2); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2 T(2^{k-1}) \end{aligned}$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^{k-1}); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k)$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^k/2); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1}$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^k/2); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2 T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2 T(2^k/2); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2 T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2 T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2 T(2^k/2); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2 T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2 T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2 T(2^k/2); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2 T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c'$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2 T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2 T(2^k/2); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2 T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2 T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2 T(2^k/2); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2 T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$, y $s_0 = t_0/2^0 = c'$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^{k-1}); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$, y $s_0 = t_0/2^0 = c' \Rightarrow$

$$s_1 = s_0 + c = c' + c$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^{k-1}); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$, y $s_0 = t_0/2^0 = c' \Rightarrow$

$$s_1 = s_0 + c = c' + c$$

$$s_2 = s_1 + c = c' + 2c$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^{k-1}); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$, y $s_0 = t_0/2^0 = c' \Rightarrow$

$$s_1 = s_0 + c = c' + c$$

$$s_2 = s_1 + c = c' + 2c$$

$$s_3 = s_2 + c = c' + 3c$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^{k-1}); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$, y $s_0 = t_0/2^0 = c' \Rightarrow$

$$s_1 = s_0 + c = c' + c$$

$$s_2 = s_1 + c = c' + 2c$$

$$s_3 = s_2 + c = c' + 3c$$

...

$$s_k = s_{k-1} + c = c' + kc$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^{k-1}); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$, y $s_0 = t_0/2^0 = c' \Rightarrow$

$$s_1 = s_0 + c = c' + c$$

$$s_2 = s_1 + c = c' + 2c$$

$$s_3 = s_2 + c = c' + 3c$$

...

$s_k = s_{k-1} + c = c' + kc \rightarrow$ inducción inmediata y no requiere demostración

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^{k-1}); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$, y $s_0 = t_0/2^0 = c' \Rightarrow$

$$s_1 = s_0 + c = c' + c$$

$$s_2 = s_1 + c = c' + 2c$$

$$s_3 = s_2 + c = c' + 3c$$

...

$s_k = s_{k-1} + c = c' + kc \rightarrow$ inducción inmediata y no requiere demostración

$$s_k = t_k/2^k$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^{k-1}); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$, y $s_0 = t_0/2^0 = c' \Rightarrow$

$$s_1 = s_0 + c = c' + c$$

$$s_2 = s_1 + c = c' + 2c$$

$$s_3 = s_2 + c = c' + 3c$$

...

$s_k = s_{k-1} + c = c' + kc \rightarrow$ inducción inmediata y no requiere demostración

$$s_k = t_k/2^k \Rightarrow t_k = 2^k(c' + ck)$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^k/2); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$, y $s_0 = t_0/2^0 = c' \Rightarrow$

$$s_1 = s_0 + c = c' + c$$

$$s_2 = s_1 + c = c' + 2c$$

$$s_3 = s_2 + c = c' + 3c$$

...

$s_k = s_{k-1} + c = c' + kc \rightarrow$ inducción inmediata y no requiere demostración

$$s_k = t_k/2^k \Rightarrow t_k = 2^k(c' + ck)$$

$$\Rightarrow t_k = T(n) = n(c' + c \lg n)$$

Merge Sort

Otra forma de resolver Merge Sort es desarrollando su ecuación de recurrencia:

$$T(n) = c \cdot n + 2T(n/2)$$

Para hacerlo más sencillo, suponga que $n = 2^k \Rightarrow k = \lg n$

Entonces la ecuación queda:

$$\begin{aligned} T(n) &= T(2^k) = c \cdot 2^k + 2T(2^k/2); \text{ con } T(1) = c' \\ &= c \cdot 2^k + 2T(2^{k-1}) \end{aligned}$$

Sea el Cambio de Variable: $t_k = T(2^k) \Rightarrow t_k = c \cdot 2^k + 2t_{k-1} \Rightarrow$

$$\frac{t_k}{2^k} = \frac{t_{k-1}}{2^{k-1}} + c \qquad s_k = t_k/2^k \Rightarrow s_k = s_{k-1} + c$$

Note que si $T(1) = c' \Rightarrow t_0 = T(2^0) = T(1) = c'$, y $s_0 = t_0/2^0 = c' \Rightarrow$

$$s_1 = s_0 + c = c' + c$$

$$s_2 = s_1 + c = c' + 2c$$

$$s_3 = s_2 + c = c' + 3c$$

...

$s_k = s_{k-1} + c = c' + kc \rightarrow$ inducción inmediata y no requiere demostración

$$s_k = t_k/2^k \Rightarrow t_k = 2^k(c' + ck)$$

$$\Rightarrow t_k = T(n) = n(c' + c \lg n) = \Theta(n \log n)$$

Merge Sort

¿Qué pasa si el tamaño de la entrada no es potencia de 2?

Merge Sort

¿Qué pasa si el tamaño de la entrada no es potencia de 2?

Por ejemplo sea $n' = n + \alpha n$, con $0 < \alpha < 1$ y $n = 2^k$

Merge Sort

¿Qué pasa si el tamaño de la entrada no es potencia de 2?

Por ejemplo sea $n' = n + \alpha n$, con $0 < \alpha < 1$ y $n = 2^k$

$$\Rightarrow 2^k = n \leq n' \leq 2n = 2^{k+1}$$

Merge Sort

¿Qué pasa si el tamaño de la entrada no es potencia de 2?

Por ejemplo sea $n' = n + \alpha n$, con $0 < \alpha < 1$ y $n = 2^k$

$$\Rightarrow 2^k = n \leq n' \leq 2n = 2^{k+1}$$

Además, como $T(n) = n(c' + c \lg n)$ es monótona creciente, sabemos que: $T(2^k + 1) \leq T(2^k + 2^k) = T(2 \cdot 2^k)$

Merge Sort

¿Qué pasa si el tamaño de la entrada no es potencia de 2?

Por ejemplo sea $n' = n + \alpha n$, con $0 < \alpha < 1$ y $n = 2^k$

$$\Rightarrow 2^k = n \leq n' \leq 2n = 2^{k+1}$$

Además, como $T(n) = n(c' + c \lg n)$ es monótona creciente, sabemos qué: $T(2^k + 1) \leq T(2^k + 2^k) = T(2 \cdot 2^k) = T(2^{k+1})$

Merge Sort

¿Qué pasa si el tamaño de la entrada no es potencia de 2?

Por ejemplo sea $n' = n + \alpha n$, con $0 < \alpha < 1$ y $n = 2^k$

$$\Rightarrow 2^k = n \leq n' \leq 2n = 2^{k+1}$$

Además, como $T(n) = n(c' + c \lg n)$ es monótona creciente, sabemos que: $T(2^k + 1) \leq T(2^k + 2^k) = T(2 \cdot 2^k) = T(2^{k+1})$

$$\Rightarrow T(n') \leq T(2n)$$

Merge Sort

¿Qué pasa si el tamaño de la entrada no es potencia de 2?

Por ejemplo sea $n' = n + \alpha n$, con $0 < \alpha < 1$ y $n = 2^k$

$$\Rightarrow 2^k = n \leq n' \leq 2n = 2^{k+1}$$

Además, como $T(n) = n(c' + c \lg n)$ es monótona creciente, sabemos que: $T(2^k + 1) \leq T(2^k + 2^k) = T(2 \cdot 2^k) = T(2^{k+1})$

$$\Rightarrow T(n') \leq T(2n) = 2n(c' + c \lg 2n)$$

Merge Sort

¿Qué pasa si el tamaño de la entrada no es potencia de 2?

Por ejemplo sea $n' = n + \alpha n$, con $0 < \alpha < 1$ y $n = 2^k$

$$\Rightarrow 2^k = n \leq n' \leq 2n = 2^{k+1}$$

Además, como $T(n) = n(c' + c \lg n)$ es monótona creciente, sabemos que: $T(2^k + 1) \leq T(2^k + 2^k) = T(2 \cdot 2^k) = T(2^{k+1})$

$$\Rightarrow T(n') \leq T(2n) = 2n(c' + c \lg 2n) = \Theta(n \log n)$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$T(n) = n^2 + 8T(n/2)$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned} T(n) &= n^2 + 8T(n/2) \\ &= n^2 + 8(n^2/4 + 8T(n/4)) \end{aligned}$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2T(n/2^2)\end{aligned}$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3))\end{aligned}$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2n^2 + 8^3T(n/2^3))\end{aligned}$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2 T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2 n^2 + 8^3 T(n/2^3) \\&\dots\end{aligned}$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$T(n) = n^2 + 8T(n/2)$$

$$= n^2 + 8(n^2/4 + 8T(n/4))$$

$$= n^2 + 2n^2 + 8^2 T(n/2^2)$$

$$= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3))$$

$$= n^2 + 2n^2 + 2^2 n^2 + 8^3 T(n/2^3))$$

...

$$= n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{k-1} n^2 + 8^k T(n/2^k))$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2n^2 + 8^3T(n/2^3)) \\&\dots \\&= n^2 + 2n^2 + 2^2n^2 + 2^3n^2 + 2^4n^2 + \dots + 2^{k-1}n^2 + 8^kT(n/2^k))\end{aligned}$$

Si $k = \lg n \Rightarrow 8^k T(n/2^k) = 8^{\lg n} T(n/2^{\lg n})$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2n^2 + 8^3T(n/2^3)) \\&\dots \\&= n^2 + 2n^2 + 2^2n^2 + 2^3n^2 + 2^4n^2 + \dots + 2^{k-1}n^2 + 8^kT(n/2^k))\end{aligned}$$

$$\text{Si } k = \lg n \Rightarrow 8^k T(n/2^k) = 8^{\lg n} T(n/2^{\lg n}) = (2^3)^{\lg n} T(1)$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2n^2 + 8^3T(n/2^3) \\&\dots \\&= n^2 + 2n^2 + 2^2n^2 + 2^3n^2 + 2^4n^2 + \dots + 2^{k-1}n^2 + 8^kT(n/2^k)\end{aligned}$$

$$\text{Si } k = \lg n \Rightarrow 8^k T(n/2^k) = 8^{\lg n} T(n/2^{\lg n}) = (2^3)^{\lg n} T(1) = 2^{\lg n^3}$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2 T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2 n^2 + 8^3 T(n/2^3)) \\&\dots \\&= n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{k-1} n^2 + 8^k T(n/2^k))\end{aligned}$$

$$\text{Si } k = \lg n \Rightarrow 8^k T(n/2^k) = 8^{\lg n} T(n/2^{\lg n}) = (2^3)^{\lg n} T(1) = 2^{\lg n^3} = n^3$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2 T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2 n^2 + 8^3 T(n/2^3) \\&\dots \\&= n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{k-1} n^2 + 8^k T(n/2^k))\end{aligned}$$

Si $k = \lg n \Rightarrow 8^k T(n/2^k) = 8^{\lg n} T(n/2^{\lg n}) = (2^3)^{\lg n} T(1) = 2^{\lg n^3} = n^3$

$$T(n) = n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{\lg n - 1} n^2 + n^3$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2 T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2 n^2 + 8^3 T(n/2^3) \\&\dots \\&= n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{k-1} n^2 + 8^k T(n/2^k)\end{aligned}$$

Si $k = \lg n \Rightarrow 8^k T(n/2^k) = 8^{\lg n} T(n/2^{\lg n}) = (2^3)^{\lg n} T(1) = 2^{\lg n^3} = n^3$

$$T(n) = n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{\lg n - 1} n^2 + n^3$$

$$\Rightarrow T(n) = \sum_{k=0}^{\lg n - 1} 2^k n^2 + n^3$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2 T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2 n^2 + 8^3 T(n/2^3) \\&\dots \\&= n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{k-1} n^2 + 8^k T(n/2^k)\end{aligned}$$

Si $k = \lg n \Rightarrow 8^k T(n/2^k) = 8^{\lg n} T(n/2^{\lg n}) = (2^3)^{\lg n} T(1) = 2^{\lg n^3} = n^3$

$$T(n) = n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{\lg n - 1} n^2 + n^3$$

$$\Rightarrow T(n) = \sum_{k=0}^{\lg n - 1} 2^k n^2 + n^3 = n^2(2^{\lg n} - 1) + n^3$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2 T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2 n^2 + 8^3 T(n/2^3) \\&\dots \\&= n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{k-1} n^2 + 8^k T(n/2^k)\end{aligned}$$

Si $k = \lg n \Rightarrow 8^k T(n/2^k) = 8^{\lg n} T(n/2^{\lg n}) = (2^3)^{\lg n} T(1) = 2^{\lg n^3} = n^3$

$$\begin{aligned}T(n) &= n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{\lg n - 1} n^2 + n^3 \\&\Rightarrow T(n) = \sum_{k=0}^{\lg n - 1} 2^k n^2 + n^3 = n^2(2^{\lg n} - 1) + n^3 = 2n^3 - n^2\end{aligned}$$

Otro Ejemplo

Resolver la recurrencia $T(n) = 8T(n/2) + n^2$; con $T(1) = 1$ y n potencia de 2.

Sol. Desarrollemos la recurrencia...

$$\begin{aligned}T(n) &= n^2 + 8T(n/2) \\&= n^2 + 8(n^2/4 + 8T(n/4)) \\&= n^2 + 2n^2 + 8^2 T(n/2^2) \\&= n^2 + 2n^2 + 8^2(n^2/2^4 + 8T(n/2^3)) \\&= n^2 + 2n^2 + 2^2 n^2 + 8^3 T(n/2^3) \\&\dots \\&= n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{k-1} n^2 + 8^k T(n/2^k)\end{aligned}$$

Si $k = \lg n \Rightarrow 8^k T(n/2^k) = 8^{\lg n} T(n/2^{\lg n}) = (2^3)^{\lg n} T(1) = 2^{\lg n^3} = n^3$

$$\begin{aligned}T(n) &= n^2 + 2n^2 + 2^2 n^2 + 2^3 n^2 + 2^4 n^2 + \dots + 2^{\lg n - 1} n^2 + n^3 \\&\Rightarrow T(n) = \sum_{k=0}^{\lg n - 1} 2^k n^2 + n^3 = n^2(2^{\lg n} - 1) + n^3 = 2n^3 - n^2 = \Theta(n^3)\end{aligned}$$

... completar el paso inductivo: PDQ. $T(2n) = 2(2n)^3 - (2n)^2$